MLFlow

Why mlflow?

When you train multiple ML models, how do you track which one is best?

That’s where MLflow comes in — It acts as the GitHub for machine learning

Real companies don’t just train models —
they track, compare, and promote the best model automatically.

2. What is MLflow?

- Tracks experiments
- Logs parameters & metrics
- Stores models
- Helps deploy models

👉 Flow:

Train → Track all the models → Save the best model → Deploy

Train Model1, Model2	Track Model1, Model2	Save best model	Deploy best model

3. Installation (I am using python3.12)

Create + Activate the environment

```
> C:\Users\hi\AppData\Local\Programs\Python\Python312\python.exe -m venv .venv

> .venv\Scripts\activate
```

Install the packages:

```
> python -m pip install --upgrade pip

> python -m pip install mlflow==3.10.1 scikit-learn==1.8.0 pandas==2.3.3 numpy==2.4.4 fastapi==0.110.0
uvicorn==0.42.0 requests skops==0.13.0 jupyter
>  python -m pip freeze -l > requirements.txt
```

Start the jupyter notebook:

```
>  python -m jupyter notebook
```

4. Basic Example (Core Demo) : Compare Multiple Models

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

# Set experiment
mlflow.set_experiment("Model_Compare_RF_vs_SVM_v1")

# Load data
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
def train_and_log(model, model_name, params):
    with mlflow.start_run(run_name=model_name):
        model.set_params(**params)
        model.fit(X_train, y_train)

        acc = model.score(X_test, y_test)

        mlflow.log_params(params)
        mlflow.log_metric("accuracy", acc)
        mlflow.set_tag("model_name", model_name)

        mlflow.sklearn.log_model(model, "model")

# Compare models
train_and_log(RandomForestClassifier(), "RandomForest", {"n_estimators": 30})
train_and_log(SVC(), "SVM", {"kernel": "rbf"})

print("Clean comparison done ")
```

👉 Key points:

- Use **experiment name**
- Use **function-based training**
- Clean logging

5. MLflow UI

👉 Open a new terminal -> Run this in terminal inside directory that contained the code (It needs internet connection):

- python -m mlflow ui

👉 Open browser:

- http://localhost:5000

👉 It shows:

- Experiments
- Accuracy comparison
- Parameters

Compare all your models visually

6. Access one of models runs through API endpoints:

Run following in same directory that contains your files:

```
> python -m mlflow models serve -m runs:/<run_id>/model --no-conda -p 5002
```

Get run_id from the model UI page.
It will look like f2193a4a8833413cb5d0a0d37401c677

This will start the server and serve the model.

Now run following code to access its API endpoints:

```
import requests
import numpy as np
from sklearn.datasets import load_iris

# Load iris data
iris = load_iris()
X = iris.data

# Take one sample (or multiple)
sample = X[0:2]   # try 2 rows

# Endpoint
url = "http://127.0.0.1:5002/invocations"

# Payload format (VERY IMPORTANT)
data = { "inputs": sample.tolist()}

# Send request
response = requests.post(url, json=data)

print("Status Code:", response.status_code)
print("Prediction:", response.json())
```

END OF STEP1

STEP2

What is done in Industry?

Structured experiments + Automated tracking + Best model promotion

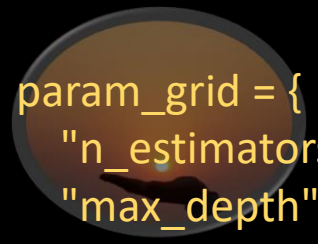
What is done in Industry? - Hyperparameter Tuning (Industry Way)

```
import mlflowimport mlflow.sklearnfrom
sklearn.ensemble import RandomForestClassifierfrom
sklearn.datasets import load_irisfrom
sklearn.model_selection import train_test_splitfrom
sklearn.model_selection import ParameterGridfrom
sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment("Hyperparameter_Tuning")

# Load data
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

best_acc = 0
best_run_id = None
```



```
param_grid = {
    "n_estimators": [10, 50,],
    "max_depth": [3, 5,],
    "min_samples_split": [2, 5],
    "criterion": ["gini",]
}
```

```
for params in ParameterGrid(param_grid):
    with mlflow.start_run():

        model = RandomForestClassifier(**params)
        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred, average="macro")

        mlflow.log_params(params)
        mlflow.log_metric("accuracy", acc)
        mlflow.log_metric("f1_score", f1)
        mlflow.sklearn.log_model(model, name="model", serialization_format="skops")

        # Track best model
        if acc > best_acc:
            best_acc = acc
            best_run_id = mlflow.active_run().info.run_id

print(f"Best Accuracy: {best_acc}")
print(f"Best Run ID: {best_run_id}")
```

👉 Key points:

- **Track best model**
- Avoid manual inspection

What is done in Industry? - Model Versioning (Correct Way)

```
model_uri = f"runs:/{best_run_id}/model"

# Register best model
registered_model = mlflow.register_model(model_uri, "IrisClassifier")

print(f"Registered Model Version: {registered_model.version}")
```

👉 Key points:

- Register **ONLY best model**
- Use **Model Registry properly**

What is done in Industry?

Optional (VERY INDUSTRY-LEVEL)

👉 Transition model stages:

```
from mlflow.tracking import MlflowClient

client = MlflowClient()

# Move to Production
client.set_registered_model_alias(
    name="IrisClassifier",
    alias="production",
    version=registered_model.version
)
```

What is done in Industry?

Step 1: Serve the Production Model

Since you already moved model to **Production**, run:

```
> python -m mlflow models serve -m models:/IrisClassifier@production --no-conda -p 5002
```

👉 This starts a REST API server

Step 2: Your Model URL

Your API endpoint will be:
`http://127.0.0.1:5002/invocations`

👉 This is the **URL to access your model**

Step 3: Send Request (IMPORTANT)

```
import requests
import json

url = "http://127.0.0.1:5002/invocations"
data = {
    "inputs": [
        [5.1, 3.5, 1.4, 0.2]
    ]
}

response = requests.post(url, json=data)
print(response.json())
```

What is done in Industry?

Experiments = Trials
Runs = Attempts
Model Registry = Production storage

Key steps:

Train models
↓
Find best run
↓
Register model
↓
Promote to Production ✅
↓
App uses: models:/Name/Production

EXTRA

Ffnbksdfd
Fdslj;f

Fl;ds